



Lecture fourteen

Standard Template Library (STL)

Ref: Herbert Schildt, Teach Yourself C++, Third Edⁿ (Chapter 14)

© **Dr. M. Mahfuzul Islam**
Professor, Dept. of CSE, BUET



An Example of Template Class

➤ Same program using **stack of character** and **stack of integer**:

```
#include <iostream>
using namespace std;
#define SIZE 10
```

```
template <class sType> class stack {
    sType stck[SIZE];
    int tos;
public:
    stack() { tos = 0; }
    void push( sType x);
    sType pop();
};
```

```
template <class sType>
void stack<sType>::push(sType ob){
    if (tos == SIZE){
        cout <<"stack is full\n";
        return;
    }
    stck[tos] = ob;
    tos++
}
```

```
template <class sType>
sType stack<sType>::pop(){
    if (tos == 0){
        cout <<"stack is empty\n";
        return 0;
    }
    tos--;
    return stck[tos];
}
```

```
int main(){
    stack<char> s1;
    stack<double> s2;
    stack<int> s3;

    s1.push('a');
    s2.push(1.1);
    s3.push(5);

    cout << s1.pop()<<' '<<s2.pop() <<' ';
    cout << s3.pop();

    return 0;
}
```



Standard Template Library (STL)

Three Important Entities of STL:

- **Containers:** A **way** that stored data is organized in memory.

Examples are stack, queue, array, vector, list, map

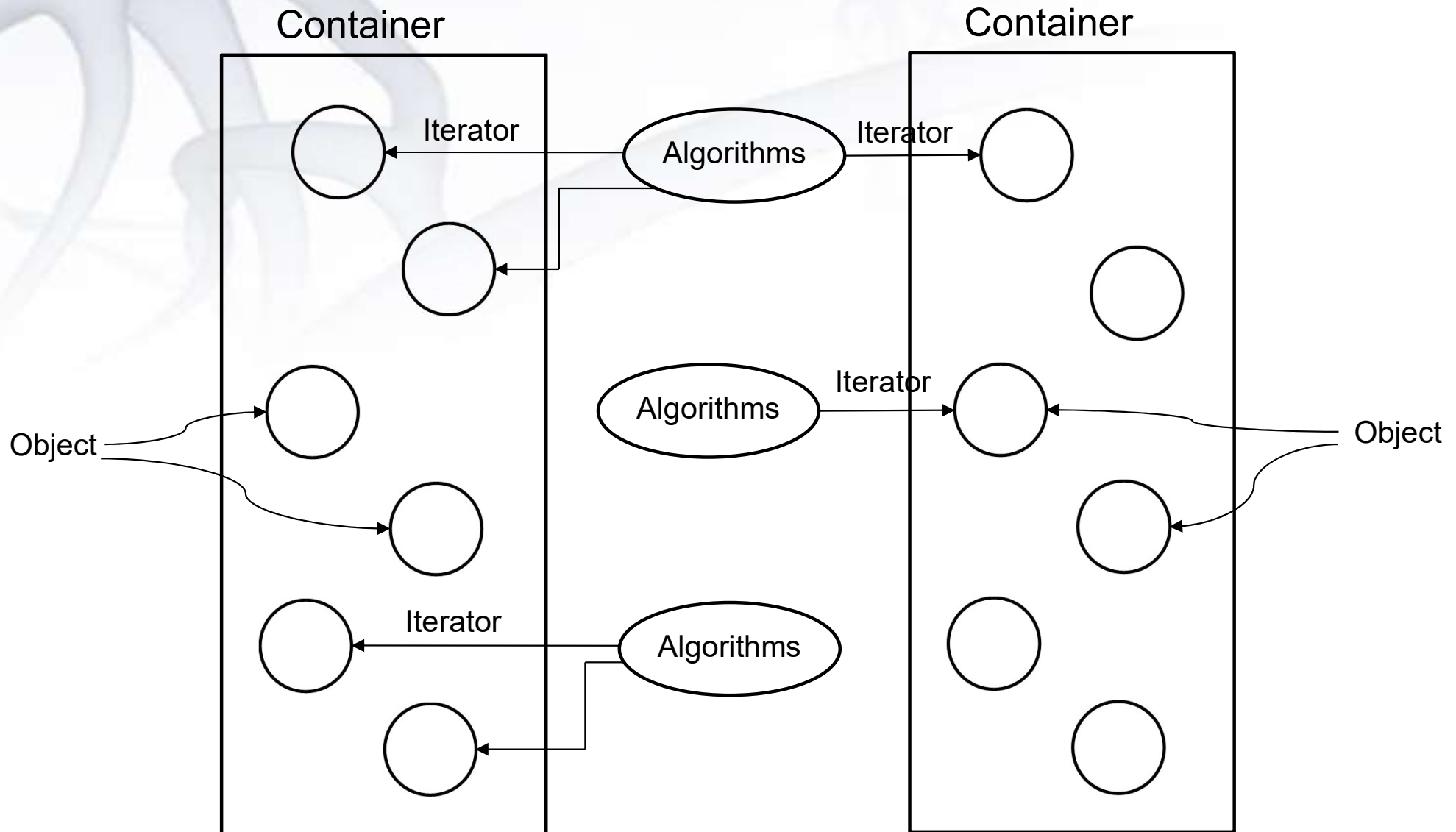
- **Algorithms:** **Standalone** procedures or functions applied to containers to process data in various ways. These functions are **not member** of container classes.

Examples initialization, sorting, searching, copying, merging, transforming

- **Iterators:** **Point** to elements in a container.



Standard Template Library (STL)





Containers

Containers – two types

- **Sequence:** Data is stored one after another. Like, array, vector, list, deque
- **Associative:** uses key to access data. There are two associative containers: set, map (also multiset, multimap)

Both use **tree** to store data – searching, insertion and deletion are easier

Container adapter is a special purpose container created from normal container with simplified interface. For example, stack, queue, priority queue

Example:

```
stack < deque <int> > aStck;
```

Note that, need to put space.

```
stack <deque<int>> aStck; //syntax error
```



Initialization and Member Function of Container

Examples:

```
vector <int> aVect;  
list <airtime> departure_list;  
set <int> iSet;  
multimap <employee> mechanists;
```

Some common member function for all containers

size()	Return number of items in the container
empty()	Return true if the container is empty
max_size()	Return the size of the largest possible container
begin()	Return an iterator to the start of the container
end()	Return an iterator to the end of the container
rbegin()	Return a reverse iterator to the end of the container
rend()	Return a reverse iterator to the start of the container



vector

vector is a smart array. The size of it is expanded or contracted when an element is entered or deleted. Need to include **<vector>**.

```
double arr[] = {3.6, 6.2, 1.0, 12.75, 4.33, 7.7, 3.3};  
vector <double> v1(arr, arr+8);  
vector <double> v2;
```

Member	Example	Explanation
push_back()	v2.push_back(2.2);	Put 2.2 at the end of array
back()	cout << v2.back();	Display last element without removing
pop_back();	cout << v2.pop_back();	Remove the last element
operator()	v1[0] = 20.5;	Conversion function to replace first value
swap()	v1.swap(v2);	Swap contents of v1 and v2
insert()	v2.insert(v2.begin()+2, 4.0);	Insert 4.0 at element 2
erase()	v2.erase(v2.begin()+2);	Erase element 2
empty()	if (v1.empty()) cout << "empty";	True if v1 is empty
size()	cout << v2.size();	Print the number of elements in v2



list

list is a doubly link list that points to both the previous and the next element. Need to include **<list>**.

```
list <int> iList;  
int arr[] = {40, 30, 20, 10};  
for (j = 0; j < 4; ++j) iList.push_back( arr[j] );
```

Member	Example	Explanation
push_back()	iList.push_back(30);	Push item on back
push_front()	iList.push_front(4);	Push item on front
front()	cout << iList.front() << " ";	Read item from front (without removing)
pop_front()	cout << iList.pop_front() << ' ';	Pop item off front
pop_back()	cout << iList.pop_back() << ' ';	Pop item off back
reverse()	iList.reverse();	Reverse iList: 10 20 30 40
merge();	iList.merge(list2);	Merge list2 into iList
unique()	iList.unique();	Remove duplicates from iList
size()	Cout << iList.size() << " ";	Print the number of elements in iList



deque

deque is like a vector in some ways and like a list in other ways. Like a vector, it support random access using [] operator; while like a list, it can be accessed at the front as well as the back. Need to include **<deque>**.

```
#include <iostream>
#include <deque>
using namespace std;

int main(){
    deque <int> deq;

    deq.push_back(30);
    deq.push_front(20);
    deq(1) = 33;

    for(int j=0; j < deq.size(); ++j)
        cout << deq[j] << " ";

    return 0;
}
```



Algorithms

Algorithms are standalone function that does something to the items of a container. Need to include <algorithm>.

```
int arr[] = {33, 22, 33, 44, 55, 44, 77, 33};  
int pattern[] = {33, 44, 55};  
int dest[11];
```

Algorithm	Example
find() find_if()	int *ptr; ptr = find(arr, arr+8, 33); //Find first 33 within the range arr to arr+8 ptr = find_if(arr, arr+8, isEven); //Find first value when isEven() returns true
count()	int n = count(arr, arr+8, 33); //count number of 33's within the arnge
sort()	sort(arr, arr+8); // sort the numbers within the range in ascending sort(arr, arr+8, greater<int>()); //takes function member to sort in desc.
search()	int* ptr; ptr = search(arr, arr+8, pattern, pattern+3); //search the range of pattern if (ptr == arr+8) cout << not found;
merge()	merge(arr, arr+8, pattern, pattern+3, dest); //merge arr & pattern and //store the rest in dest



Algorithms

```
double inch[] = {3.6, 6.2, 1.0, 12.75, 4.33, 7.7, 3.3};  
double cm[8];
```

```
double in2cm(double in){  
    return in * 2.54;  
}
```

```
double print2cm(double in){  
    cout << in * 2.54 << " ";  
}
```

Algorithm	Example
for_each()	For_each(inch, inch+8, print2cm); // for each element run print2cm
transform()	transform(inch, inch+8, cm, in2cm); //for each element within the range, run in2cm and store values in cm



Iterators

Use of pointer to perform an operation on all elements of a container.

```
double inch[] = {3.6, 6.2, 1.0, 12.75, 4.33};  
double* ptr = inch;
```

```
For( int j = 0; j < 5; ++j )  
    cout << *ptr++ << " ";
```

Defining iterator

```
list <int> iList;  
list <int>::iterator iter;  
  
for( iter = iList.begin(); iter != iList.end(); iter++)  
    cout << *iter << " ";
```

Problems of Using Pointer:

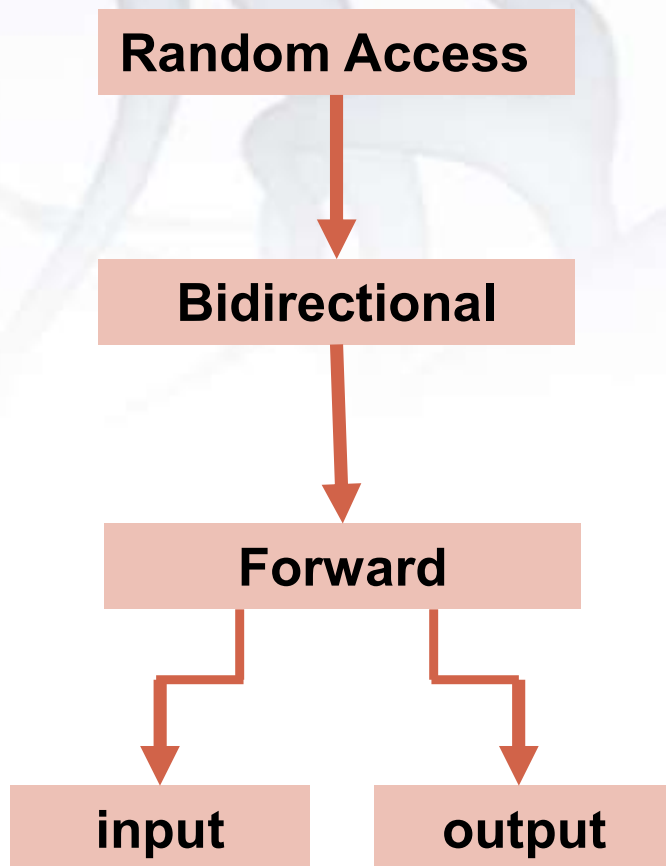
- If the items stored in a container are not placed contiguously, then handling the pointer becomes complicated.
- Inserting and erasing something from middle of container, requires to overload ++ and * operator.

Matching algorithm with container:

- When an iterator is declared for a list, it automatically becomes bidirectional.
- An iterator to a vector or a deque is automatically created as a random-access iterator.



Iterators



Input: when an algorithm steps forward and read items, e.g., cin

Output: when an algorithm steps forward and write items, e.g., cout

Forward: when an algorithm steps forward and either read from or write to the container.

Bidirectional: when an algorithm steps both forward and backward through a container.

Random access: when an algorithm accesses any item instantly, without stepping along it like array.



Iterators

A sample program using iterator

```
#include <iostream>
#include <algorithm>
#include <list>
using namespace std;

int main(){
    list <int> iList(5);
    list <int>::iterator iter;
    int data = 0;

    for ( iter = iList.begin(); iter != iList.end();
        ++iter )
        *iter = data += 2;

    iter = find(iList.begin(), iList.end(), 8);
    if ( iter != iList.end() )
        cout << "8 found.";
    else cout << "Not found";

    return 0;
}
```

A sample program with reverse iterator

```
#include <iostream>
#include <list>
using namespace std;

int main(){
    int arr[] = {2, 4, 6, 8, 10}
    list <int> iList;

    for ( int j=0; j < 5; ++j)
        iList.pushback( arr[j] );

    list <int>:: reverse_iterator revit;

    revit = iList.rbegin();
    while ( revit != iList.rend() )
        cout << *revit << " ";

    return 0;
}
```



Associative Container: **map**

- The **map class** supports an **associative container** in which **unique keys** are **mapped** with **values**.
- The value in first contains the **key** and the value in second contains the **value**.
- The **key/value pair** stored are:

A	0
B	1
C	2

The program for the above pair:

```
#include <iostream>
#include <map>
using namespace std;

int main(){
    map<char, int> m;
    int i;

    for(i=0; i<10; ++i)
        m.insert(pair<char, int>('A'+i, i));

    char ch;
```

```
cout << "Enter key:";
cin >> ch;
map<char, int>:: iterator p;

p = m.find(ch);
if (p !=m.end())
    cout << p->second;
else
    cout << "key is not in map\n";

    return 0;
}
```

- The function **make_pair()** can be used instead of pair.

m.insert(make_pair((char) ('A'+i), i));



Associative Container: **set**

```
#include <iostream>
#include <set>
#include <string>
using namespace std;

int main(){
    string names[] = {"Juanita", "Robert", "Mary", "Amanda", "Marie"};
    set < string, less<string> > nameSet(names, Names+5);
    set < string, less<string> >:: iterator iter;

    nameSet.insert("Larry"); // Larry added
    nameSet.insert("Robert"); //no effect, already in set
    nameSet.erase("Mary"); // Mary deleted
    iter = nameSet.begin();
    while ( iter != nameSet.end() )
        cout << *iter++ << " ";
    string sname;
    cout << "Enter a name: ";
    cin >> sname;
    iter = nameSet.find(sname);
    if ( iter == nameSet.end())
        cout << "Not found";

    return 0;
}
```